

BACSE202 – Database Systems

Query Optimization: Query Trees and Heuristic Optimization

Module 3. Built from the course slides (BACSE202-DBS-Module-3, slides 265–283) and Elmasri & Navathe, Ch. 18–19.

Contents

1 Introduction to Query Processing	1
1.1 Basic steps in query processing (slide 267)	1
1.2 Process for heuristic optimization (slide 268)	2
2 Query Representation: Query Tree and Query Graph	2
2.1 Translation first: query blocks (slide 270)	2
2.2 Example Q2: query tree and query graph (slides 272–274)	3
3 Heuristic Optimization of Query Trees	3
3.1 Steps in optimizing a query tree (slide 276)	3
3.2 Worked example (slides 276–279, Fig. 18.5)	4
4 Transformation Rules for Relational Algebra	5
5 The Heuristic Optimization Algorithm (Summary)	7
6 Practice: Optimize a Query	7
7 Errors in the Slides	7
8 Quick Revision	8

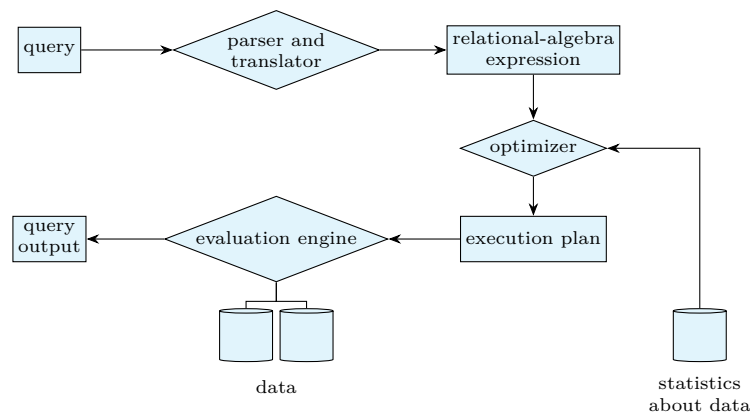
1 Introduction to Query Processing

Definition (slide 266)

Query optimization is the process of **choosing a suitable execution strategy** for processing a query. The query is held internally in one of two forms: a **query tree** or a **query graph**.

- The same SQL query can be evaluated in **many** equivalent ways, and they differ enormously in cost (disk I/O, CPU, memory). The optimizer picks a good one.
- The optimizer does not really find the *optimal* plan (that search is too expensive). It finds a **reasonably efficient** one, so “optimization” is a slight misnomer.

1.1 Basic steps in query processing (slide 267)



Step	What happens
1. Parsing and translation	The scanner finds tokens (keywords, names), the parser checks syntax, and the query is validated (do the relations and attributes exist?). It is then translated into an internal form: a relational algebra expression, i.e. a query tree or graph.
2. Optimization	The optimizer rewrites the expression and chooses algorithms and access paths (indexes, sort order). It uses statistics about the data (number of tuples, distinct values, index availability) to produce an execution plan .
3. Evaluation	The evaluation engine (runtime database processor) runs the plan against the data and returns the query output.

1.2 Process for heuristic optimization (slide 268)

1. The **parser** of a high-level query generates an **initial internal representation** (the canonical query tree).
2. **Heuristic rules** are applied to optimize this internal representation.
3. A **query execution plan** is generated to execute groups of operations, based on the **access paths** available on the files involved in the query.

The main heuristic

Apply first the operations that **reduce the size of intermediate results**. For example, apply **SELECT** (σ) and **PROJECT** (π) **before** the **JOIN** or other binary operations.

Why it works: a binary operation such as $\times$ or $\bowtie$ is expensive in proportion to the *sizes* of its inputs. σ shrinks the number of **rows**, and π shrinks the **width** of each row (so more rows fit per block). Doing them early means every later operation handles far less data.

Heuristic (rule-based) optimization	Cost-based optimization
Applies transformation rules that usually improve the query tree, without estimating costs.	Estimates the cost of alternative plans from catalog statistics and picks the cheapest.
Fast; good for simple queries; always improves the canonical tree.	More accurate but more work; used by real DBMSs, usually <i>after</i> heuristic rewriting.

(The slides cover heuristic optimization; cost-based optimization is shown here for contrast only.)

2 Query Representation: Query Tree and Query Graph

Query tree	Query graph
A tree data structure that corresponds to a relational algebra expression.	A graph data structure that corresponds to a relational calculus expression.
Leaf nodes = input relations; internal nodes = RA operations.	Relation nodes (single circles) and constant nodes (double circles); edges = selection and join conditions; attributes to retrieve shown in [].
Execution: run an internal node as soon as its operands are available, and replace that node by the resulting relation. Execution ends when the root is executed.	Does not indicate an order in which to perform operations.
Many trees for one query (fixes an order of evaluation).	Only a single graph for each query.
Preferred by optimizers, because the order is explicit.	Less used, because it cannot express an execution order.

2.1 Translation first: query blocks (slide 270)

Before optimizing, the SQL query is translated into RA (see the separate *SQL to Relational Algebra* notes).

- A **query block** is the basic unit that is translated into algebraic operators and optimized. It contains a single SELECT-FROM-WHERE expression, plus GROUP BY and HAVING if they are part of the block.
- **Nested queries** are identified as **separate query blocks**. Aggregate operators must be included in the **extended algebra**.
- Example (slide 271): `SELECT LNAME, FNAME FROM EMPLOYEE WHERE SALARY > (SELECT MAX(SALARY) FROM EMPLOYEE WHERE DNO = 5)` splits into the inner block $\mathcal{F}_{MAX\ SALARY}(\sigma_{DNO=5}(EMPLOYEE))$, giving a constant c , and the outer block $\pi_{LNAME,FNAME}(\sigma_{SALARY>c}(EMPLOYEE))$. Each block is optimized separately.

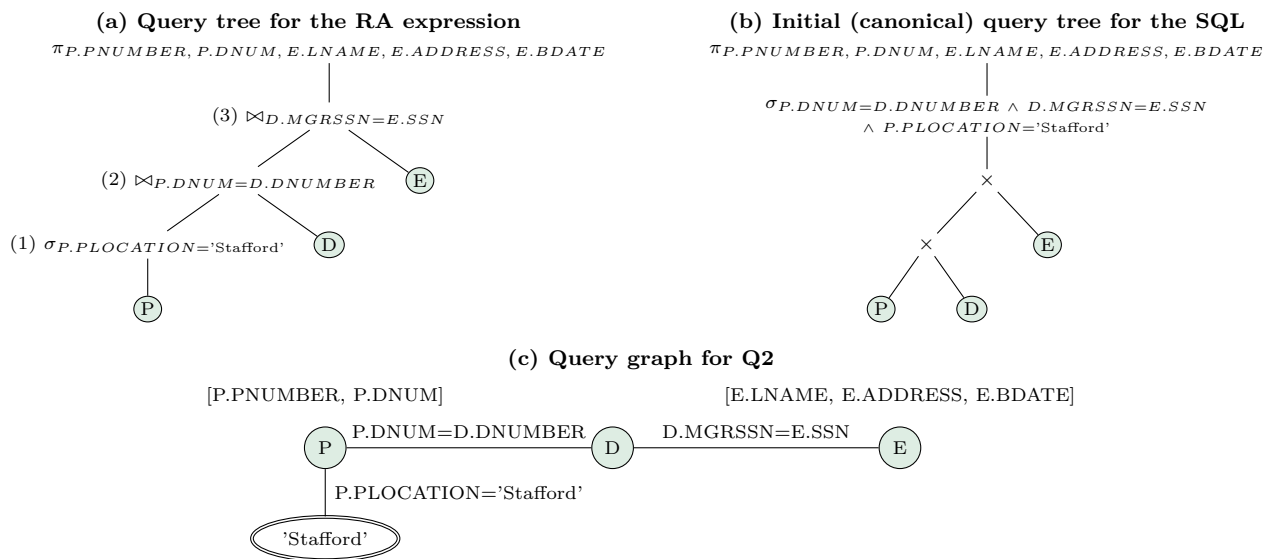
2.2 Example Q2: query tree and query graph (slides 272–274)

Q2

For every project located in 'Stafford', retrieve the project number, the controlling department number, and the department manager's last name, address and birth date.

```
SELECT P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE
FROM   PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E
WHERE  P.DNUM = D.DNUMBER AND D.MGRSSN = E.SSN AND P.PLOCATION = 'Stafford';
```

$$\pi_{PNUMBER, DNUM, LNAME, ADDRESS, BDATE} \left(\left((\sigma_{PLOCATION='Stafford'}(PROJECT)) \right) \right. \\ \left. \bowtie_{DNUM=DNUMBER} DEPARTMENT \right) \bowtie_{MGRSSN=SSN} EMPLOYEE$$



- Tree (a) executes bottom-up in the numbered order: (1) select the Stafford projects, (2) join with DEPARTMENT, (3) join with EMPLOYEE, then project.
- Tree (b) is what the parser produces directly: $\times$ of all FROM tables, one big σ , one π . It is **correct but very inefficient**. Heuristic optimization turns (b) into something like (a).

3 Heuristic Optimization of Query Trees

Idea (slide 275)

- The same query can correspond to **many different relational algebra expressions**, and hence **many different query trees**, all giving the same answer.
- The task of heuristic optimization is to transform the initial tree into a **final query tree that is efficient to execute**, using transformation rules that are guaranteed to preserve the result (Section 4).

3.1 Steps in optimizing a query tree (slide 276)

1. Move **SELECT** operations down the query tree (as close to the leaves as possible).
2. Apply the **more restrictive SELECT operation first** (reorder the leaves so that the relations with the smallest selections are processed first).
3. Replace **CARTESIAN PRODUCT** and **SELECT with JOIN** operations.
4. Move **PROJECT** operations down the tree (keep only the attributes needed higher up).

3.2 Worked example (slides 276–279, Fig. 18.5)

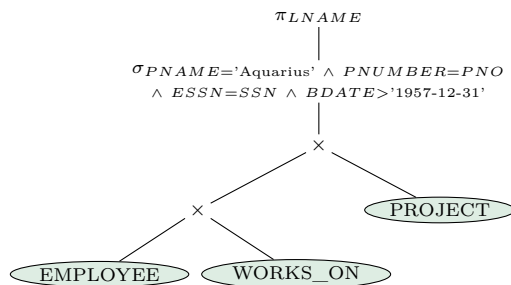
Query Q

Find the last names of employees born after 1957 who work on a project named 'Aquarius'.

```
SELECT LNAME
FROM   EMPLOYEE, WORKS_ON, PROJECT
WHERE  PNAME = 'Aquarius' AND PNUMBER = PNO AND ESSN = SSN AND BDATE > '1957-12-31';
```

Conditions: $PNAME = 'Aquarius'$ and $BDATE > '1957-12-31'$ are **selections** (one table each). $PNUMBER = PNO$ (PROJECT-WORKS_ON) and $ESSN = SSN$ (WORKS_ON-EMPLOYEE) are **join conditions**.

(a) Initial (canonical) query tree

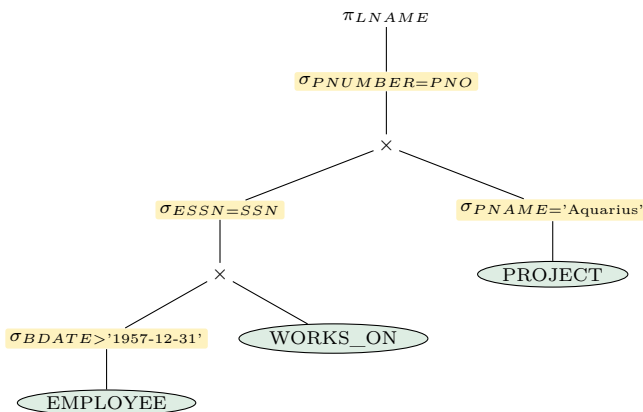


$$\pi_{LNAME}(\sigma_C((E \times W) \times P))$$

($E = \text{EMPLOYEE}$, $W = \text{WORKS_ON}$, $P = \text{PROJECT}$, $C =$ the whole WHERE clause.) The FROM tables are combined with $\times$ in the order written, then one σ with the whole WHERE clause, then π .

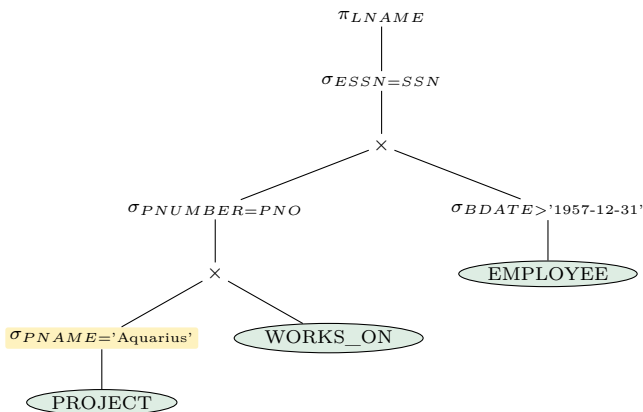
Cost: with the textbook COMPANY data (8 employees, 16 WORKS_ON rows, 6 projects), the product alone produces $8 \times 16 \times 6 = 768$ tuples, and each is as wide as all three relations together. In a real database with thousands of rows per table this is millions or billions of tuples.

(b) Step 1: move SELECT operations down



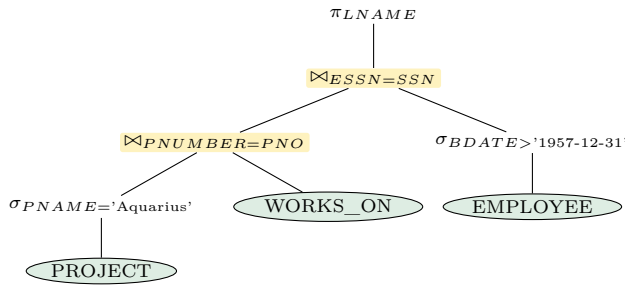
- Split the conjunctive σ into four separate selections (**cascade of σ** , rule 1).
- Push each one down as far as its attributes allow (rules 2, 7).
- σ_{BDATE} only uses EMPLOYEE, so it goes directly above EMPLOYEE. σ_{PNAME} goes directly above PROJECT.
- $\sigma_{ESSN=SSN}$ needs EMPLOYEE and WORKS_ON, so it sits just above their $\times$. $\sigma_{PNUMBER=PNO}$ needs WORKS_ON and PROJECT, so it stays above the top $\times$.

(c) Step 2: apply the more restrictive SELECT first



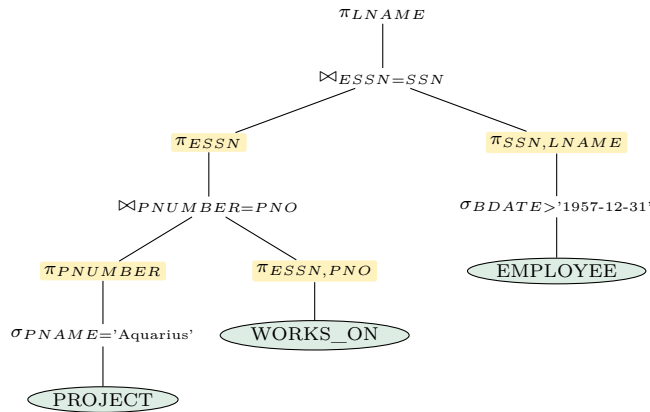
- $\sigma_{PNAME='Aquarius'}$ is the **most restrictive**: PNAME is (nearly) unique, so it returns about **one** project. $\sigma_{BDATE>'1957-12-31'}$ may return many employees.
- So the leaves are **rearranged**: PROJECT is processed first and combined with WORKS_ON, and EMPLOYEE comes last. Rules 5, 6 and 10 (commutativity and associativity) allow the reordering.
- The rearrangement also avoids a **pure Cartesian** product: every $\times$ now has a join condition directly above it.

(d) Step 3: replace CARTESIAN PRODUCT and SELECT with JOIN



- Every σ_θ that sits directly above a $\times$ with a join condition θ becomes a join: $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$ (rule 4).
- Joins can use efficient algorithms (index lookups, sort-merge, hash join) and never build the full product.

(e) Step 4: move PROJECT operations down



- Above each node keep only the attributes needed **later**: in join conditions or the final result (rules 3, 8).
- From PROJECT only PNUMBER is needed (for $PNUMBER = PNO$). From WORKS_ON only ESSN, PNO. After that join, only ESSN. From EMPLOYEE only SSN (join) and LNAME (result).
- Intermediate tuples become much **narrower**.

Final optimized expression

$$\pi_{LNAME} \left(\pi_{ESSN} \left(\pi_{PNUMBER} \left(\sigma_{PNAME='Aquarius'}(PROJECT) \right) \right. \right. \\ \left. \left. \bowtie_{PNUMBER=PNO} \pi_{ESSN,PNO}(WORKS_ON) \right) \right. \\ \left. \left. \bowtie_{ESSN=SSN} \pi_{SSN,LNAME}(\sigma_{BDATE > '1957-12-31'}(EMPLOYEE)) \right) \right)$$

Compare with the canonical $\pi_{LNAME}(\sigma_C(EMPLOYEE \times WORKS_ON \times PROJECT))$: same result, far less work.

4 Transformation Rules for Relational Algebra

Two RA expressions are **equivalent** if they produce the same set of tuples on *every* legal database instance. These rules (slides 280–283) are what make each optimization step legal. E , E_1 , E_2 , E_3 are RA expressions, θ are conditions and L are attribute lists.

Rules on selection and projection

1. **Cascade of σ .** A conjunctive selection can be broken into a sequence of individual selections:

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. **Commutativity of σ .**

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. **Cascade of π .** Only the final projection in a sequence is needed; the others can be omitted:

$$\pi_{L_1}(\pi_{L_2}(\dots(\pi_{L_n}(E))\dots)) = \pi_{L_1}(E) \quad (L_1 \subseteq L_2 \subseteq \dots \subseteq L_n)$$

Used for: rule 1 splits the big WHERE σ into pieces, and rule 2 lets each piece slide past the others on its way down.

Rules on joins and products

4. Selections combine with Cartesian products and theta joins.

$$\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2 \qquad \sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

5. Theta join (and $\times$) is commutative.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

6. Natural join is associative.

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

Theta joins are associative in the following manner, where θ_2 involves attributes from E_2 and E_3 only:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

Used for: rule 4 is step 3 (product + selection $\rightarrow$ join). Rules 5 and 6 allow the join order to be rearranged (step 2).

Distribution over joins

7. σ distributes over the theta join under two conditions:

(a) when all attributes in θ_0 involve only the attributes of **one** of the expressions (E_1) being joined:

$$\sigma_{\theta_0}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_\theta E_2$$

(b) when θ_1 involves only attributes of E_1 and θ_2 only attributes of E_2 :

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_\theta (\sigma_{\theta_2}(E_2))$$

8. π distributes over the theta join. Let L_1 and L_2 be attributes of E_1 and E_2 .

(a) If the join condition θ involves only attributes in $L_1 \cup L_2$:

$$\pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\pi_{L_1}(E_1)) \bowtie_\theta (\pi_{L_2}(E_2))$$

(b) In general, let L_3 be the attributes of E_1 used in θ but not in $L_1 \cup L_2$, and L_4 the attributes of E_2 used in θ but not in $L_1 \cup L_2$. Then:

$$\pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \pi_{L_1 \cup L_2}((\pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\pi_{L_2 \cup L_4}(E_2)))$$

Used for: rule 7 pushes σ down (step 1). Rule 8 pushes π down (step 4). In 8(b) the join attributes L_3, L_4 must be kept below the join and dropped only above it, which is why tree (e) keeps PNUMBER, PNO and SSN until their joins are done.

Rules on set operations

9. Union and intersection are commutative (set difference is **not**):

$$E_1 \cup E_2 = E_2 \cup E_1 \qquad E_1 \cap E_2 = E_2 \cap E_1$$

10. Union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3) \qquad (E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. σ distributes over $\cup$, $\cap$ and $-$. For $\ominus \in \{\cup, \cap, -\}$: $\sigma_P(E_1 \ominus E_2) = \sigma_P(E_1) \ominus \sigma_P(E_2)$. For $\cap$ and $-$ it is enough to select on E_1 only:

$$\sigma_P(E_1 - E_2) = \sigma_P(E_1) - E_2 = \sigma_P(E_1) - \sigma_P(E_2)$$

(This does **not** hold for $\cup$: $\sigma_P(E_1 \cup E_2) \neq \sigma_P(E_1) \cup \sigma_P(E_2)$.)

12. π distributes over $\cup$.

$$\pi_L(E_1 \cup E_2) = (\pi_L(E_1)) \cup (\pi_L(E_2))$$

(π does **not** distribute over $\cap$ or $-$ in general.)

Why π does not distribute over $-$

$E_1 = \{(1, a)\}$, $E_2 = \{(1, b)\}$ with attributes (A, B) . Then $\pi_A(E_1 - E_2) = \pi_A(\{(1, a)\}) = \{1\}$, but $\pi_A(E_1) - \pi_A(E_2) = \{1\} - \{1\} = \emptyset$.

5 The Heuristic Optimization Algorithm (Summary)

Elmasri & Navathe's outline, which is the four slide steps written as rule applications:

1. **Break up** conjunctive selections into a cascade of σ (rule 1). *This gives freedom to move each piece separately.*
2. **Move each σ as far down** the tree as its attributes allow (rules 2, 4, 7, 11).
3. **Rearrange the leaf nodes** so that the **most restrictive** selections are executed first, i.e. those producing the fewest tuples or the smallest relation. Avoid orders that lead to a pure Cartesian product (rules 5, 6, 9, 10).
4. **Combine** a $\times$ with a σ above it that holds a join condition into a **join** (rule 4).
5. **Break up and move lists of projection attributes down** the tree as far as possible, creating new π operations where needed (rules 3, 8, 12).
6. **Identify subtrees** that are groups of operations executable by a single algorithm (for example a $\sigma + \bowtie + \pi$ pipeline) and generate the execution plan.

One-line summary

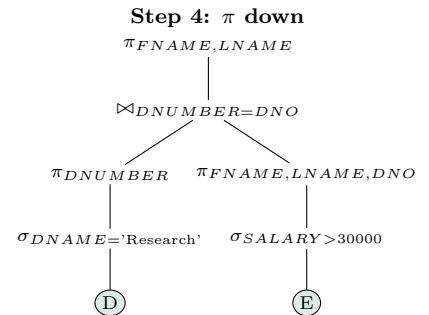
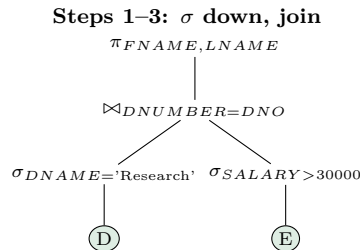
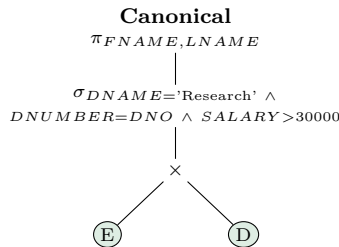
Selections down, restrictive first, products become joins, projections down. Every step shrinks intermediate results, either fewer rows (σ , joins instead of $\times$) or narrower rows (π).

6 Practice: Optimize a Query

Exercise

SELECT E.FNAME, E.LNAME FROM EMPLOYEE E, DEPARTMENT D WHERE D.DNAME = 'Research' AND D.DNUMBER = E.DNO AND E.SALARY > 30000;

Draw the canonical tree and optimize it step by step.



- $\sigma_{DNAME='Research'}$ gives **one** department, so D is placed first (the most restrictive selection).
- With the COMPANY data (8 employees, 3 departments), the canonical product has $8 \times 3 = 24$ tuples. After optimization the join combines **1** department tuple with the **4** employees earning over 30000 (Wong, Wallace, Narayan, Borg), and **2** tuples come out: Franklin Wong and Ramesh Narayan. Smith earns exactly 30000, so he fails $SALARY > 30000$.
- Final:

$$\pi_{FNAME, LNAME} \left(\pi_{DNUMBER} \left(\sigma_{DNAME='Research'} (DEPARTMENT) \right) \right. \\ \left. \bowtie_{DNUMBER=DNO} \pi_{FNAME, LNAME, DNO} \left(\sigma_{SALARY>30000} (EMPLOYEE) \right) \right)$$

7 Errors in the Slides

Corrections

Slide	Issue → correct version
276	“PNMUBER” → PNUMBER . The SQL writes ‘AQUARIUS’ and ‘1957-12-31’, while the trees (Fig. 18.5) write ‘Aquarius’ and ‘DEC-31-1957’. They mean the same values.
281	Rule 4: “ $\sigma_{\theta_1}(E_1 \bowtie_{\sigma_{\theta_2}} E_2)$ ” has a stray σ in the join subscript → $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$.
283	Rule 11 shows only the $-$ case. σ also distributes over $\cup$ and $\cap$, but the shortcut $\sigma_P(E_1) \ominus E_2$ is valid only for $-$ and $\cap$, not for $\cup$.
272	“P.NUMBER” → P.PNUMBER ; “ ‘ STAFFORD ’ ” has stray spaces.

8 Quick Revision

- **Query optimization** = choosing a suitable execution strategy. Internal forms: **query tree** (RA, leaves = relations, internal nodes = operators, many per query) and **query graph** (relational calculus, no order, one per query).
- **Query processing**: parsing and translation → optimization (uses statistics) → evaluation. **Query block** = one SELECT-FROM-WHERE (+ GROUP BY/HAVING), and nested queries are separate blocks.
- **Heuristic process**: parser builds the initial tree → heuristic rules improve it → execution plan based on access paths.
- **Main heuristic**: apply σ and π **before** joins and other binary operations, to reduce intermediate results.
- **Four steps**: (1) move σ down, (2) most restrictive σ first, (3) $\times + \sigma \rightarrow \bowtie$, (4) move π down.
- **Key rules**: cascade/commute of σ (1, 2); cascade of π (3); $\sigma(\times) = \bowtie$ (4); $\bowtie$ commutative and associative (5, 6); σ and π distribute over $\bowtie$ (7, 8); $\cup, \cap$ commutative and associative, $-$ is not (9, 10); σ distributes over $\cup, \cap, -$ (11); π distributes over $\cup$ (12).
- **Aquarius example**: canonical $\pi(\sigma(E \times W \times P)) \rightarrow \pi_{LNAME}(\pi_{ESSN}(\pi_{PNUMBER}(\sigma_{PNAME}(P)) \bowtie \pi_{ESSN,PNO}(W)) \bowtie \pi_{SSN,LNAME}(\sigma_{BDATE}(E)))$.